

Universidad Lead

2026 - Computación Paralela y Distribuida

Tarea 3:

Procesamiento de datos con Polars

Docente: Johansell Villalobos Cubillo

Integrantes:

Sebastián Blanco

Daniel Lara

1. Introducción

Este informe expone el desarrollo de un pipeline integral de análisis de datos y aprendizaje automático construido con Polars, una biblioteca moderna diseñada para el procesamiento eficiente de datos tabulares. Polars se fundamenta en Apache Arrow y sobresale por su ejecución paralela, su motor de consultas optimizado y su capacidad para operar mediante evaluación diferida o Lazy Execution. Más allá de implementar el flujo de procesamiento y modelado, el trabajo contrasta el desempeño de Polars con el de Pandas en las distintas fases del pipeline, con el objetivo de evaluar sus beneficios computacionales dentro de un escenario práctico de ciencia de datos.

El problema planteado consiste en una tarea de clasificación binaria: determinar si un vuelo arriba con retraso o no. Con este propósito se empleó el dataset Flight Delay Prediction de Kaggle, integrado por 107 833 registros y 10 variables asociadas a vuelos de Tunisair y aerolíneas vinculadas. La variable objetivo binaria, denominada retrasado, se derivó de la variable continua target —que indica los minutos de retraso en la llegada— tomando como referencia un umbral de 15 minutos. El análisis se enfoca en dos ejes fundamentales: por una parte, la calidad predictiva de los modelos a través de métricas como Accuracy, F1 Score y AUC; y, por otra, la eficiencia computacional del pipeline mediante la comparación de los tiempos de ejecución entre Polars y Pandas.

2. Descripción del Dataset

Fuente: Kaggle — Flight Delay Prediction (abderrahimalakouche/flight-delay-prediction). Datos operativos de la aerolínea Tunisair, descargados mediante kagglehub.

El dataset original contiene 107 833 registros y 10 columnas, sin valores faltantes. Las variables disponibles son:

- DATOP: fecha de operación del vuelo.
- FLTID: identificador del vuelo (incluye el código de aerolínea).
- DEPSTN / ARRSTN: aeropuertos de origen y destino.
- STD / STA: hora de salida y llegada programadas.
- STATUS: estado del vuelo (ATA, SCH, DEP, RTR, DEL).
- AC: tipo/matrícula de la aeronave.

- target: minutos de retraso (variable continua de la que se deriva la clase).

Análisis de la variable objetivo (target): media de 48,7 minutos, mediana de 14, desviación estándar de 117,1 y un máximo de 3 451 minutos, lo que evidencia una distribución muy sesgada a la derecha con outliers extremos. La distribución de STATUS muestra que 93 679 vuelos tienen estado ATA (aterrizados, con retraso real), 13 242 son SCH (solo programados) y el resto son estados marginales.

Decisiones de limpieza documentadas: (1) se conservaron únicamente los vuelos con STATUS = ATA, ya que los SCH no tienen retraso real medido; (2) se eliminaron los outliers extremos con target > 300 minutos (3 380 registros, 3,61 % del total), que corresponden a cancelaciones reprogramadas y no a retrasos operativos normales; (3) se construyó la variable binaria retrasado con el umbral de 15 minutos. Tras la limpieza quedaron 90 299 registros con un balance de clases equilibrado: 43 665 vuelos a tiempo (48,4 %) y 46 634 retrasados (51,6 %).

3. Metodología y Pipeline de Datos

Todo el procesamiento previo al modelado se realizó exclusivamente con Polars, aprovechando sus expresiones vectorizadas. El pipeline se organizó en análisis exploratorio e ingeniería de características.

3.1 Análisis Exploratorio (EDA)

Se calcularon estadísticas descriptivas, conteo de nulos y distribuciones con Polars. La Figura 1 muestra la distribución del retraso (recortada a 300 minutos para visualización), donde se aprecia la fuerte concentración de vuelos cerca del umbral de 15 minutos.

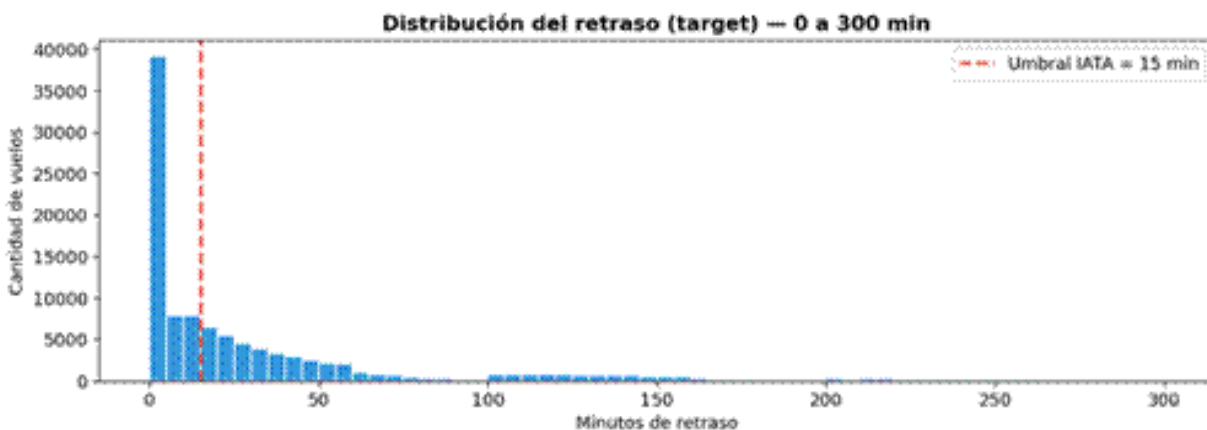


Figura 1. Distribución de la variable target (minutos de retraso, 0–300). La línea roja marca el umbral IATA de 15 min.

Para el análisis de correlaciones se derivaron variables numéricas temporales (mes, día de la semana, hora de salida y hora de llegada) parseando las columnas de fecha/hora, que venían en dos formatos mezclados (HH.MM.SS y HH:MM:SS). La Figura 2 presenta la matriz de correlación de Pearson entre las variables numéricas.

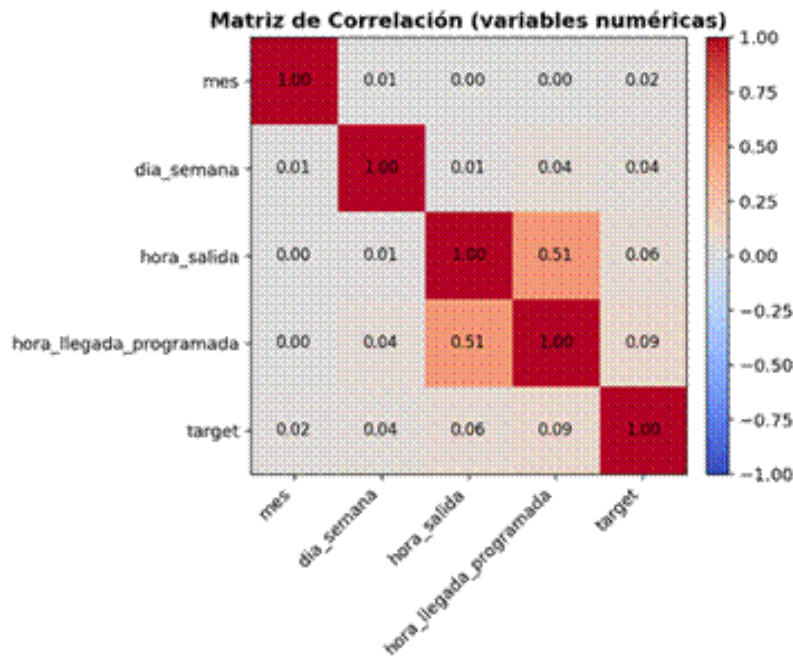


Figura 2. Matriz de correlación entre variables numéricas. Las correlaciones lineales con el retraso son débiles.

3.2 Ingeniería de Características

El pipeline de transformación con Polars incluyó filtrado de registros, manejo de datos faltantes, creación de nuevas características, dos joins y agregaciones con `group_by`:

- Código de aerolínea: extraído de los primeros 2 caracteres de FLTID.
- Modelo de aeronave: extraído de AC mediante expresión regular (p.ej. 32A, 320I, 736I, que concentran la mayoría de la flota).
- Retraso promedio histórico por aeropuerto de salida y por ruta (origen→destino), calculados con `group_by` y unidos al dataset mediante `left join`.
- Manejo de nulos: los promedios faltantes (rutas con muy pocos vuelos) se rellenaron con la mediana global del retraso (17 minutos).
- Encoding: las variables categóricas (`codigo_aerolinea`, `modelo_aeronave`) se codificaron a enteros con `Categorical → to_physical()`.

La Figura 3 confirma el balance de clases tras la limpieza. El dataset final para el modelo quedó con 90 299 filas y 9 columnas (8 features + objetivo), sin valores nulos.

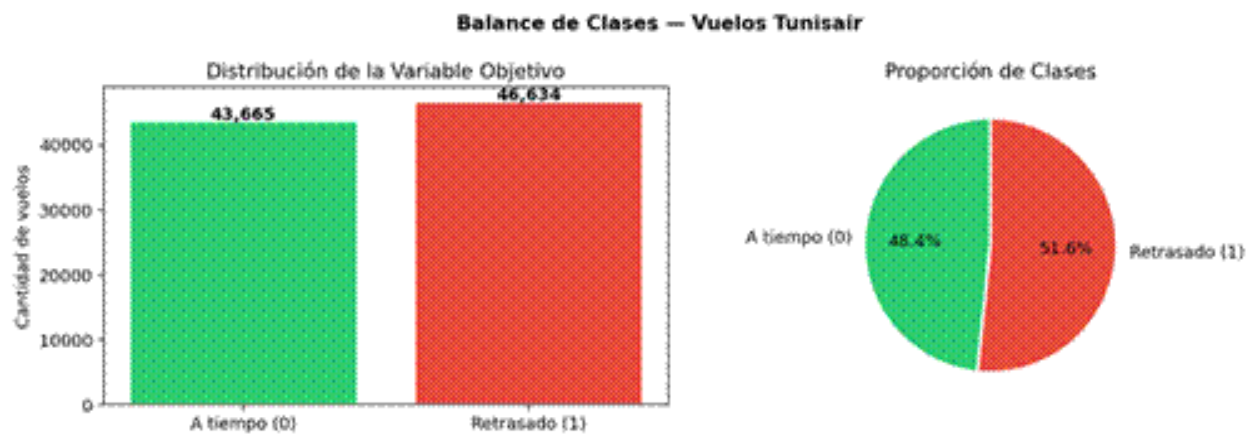


Figura 3. Balance de clases de la variable objetivo: 48,4 % a tiempo y 51,6 % retrasados.

4. Resultados de Machine Learning

Se entrenaron y evaluaron tres modelos con una división estratificada 80/20 (72 239 registros de entrenamiento y 18 060 de prueba, semilla fija 42). La Tabla 1 resume las métricas y el tiempo de entrenamiento de cada modelo.

Modelo	Accuracy	F1	AUC	Tiempo (s)
Regresión Logística	0.6446	0.6598	0.6953	0.37
Random Forest	0.6645	0.6762	0.7152	0.55
XGBoost	0.6996	0.7111	0.7681	0.27

Tabla 1. Comparación de modelos. XGBoost obtiene el mejor desempeño en todas las métricas y, además, es el más rápido de entrenar.

Matrices de confusión sobre el conjunto de prueba (VP / FN / FP / VN):

- Regresión Logística: VP=6 225, FN=3 102, FP=3 317, VN=5 416.
- Random Forest: VP=6 326, FN=3 001, FP=3 058, VN=5 675.
- XGBoost: VP=6 677, FN=2 650, FP=2 775, VN=5 958.

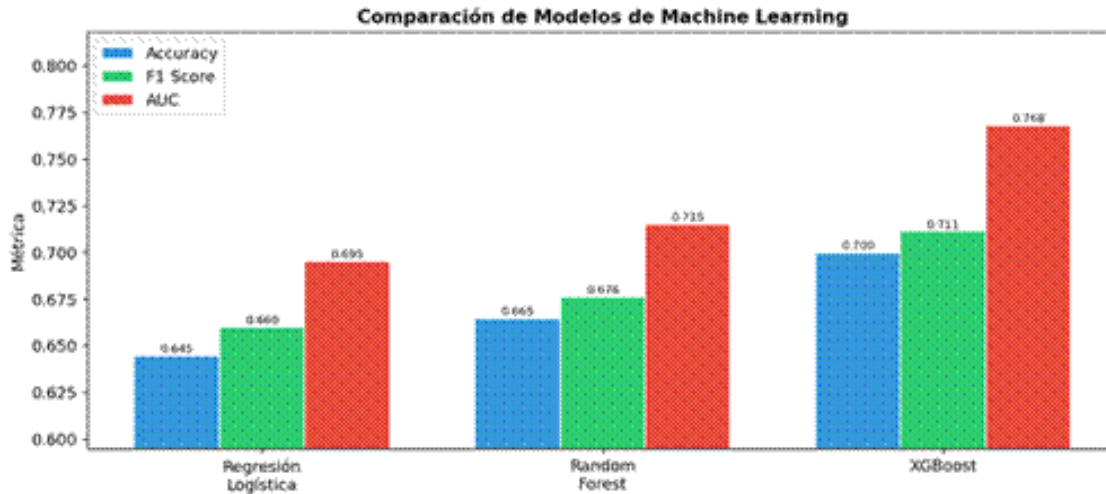


Figura 4. Comparación de Accuracy, F1 y AUC por modelo. XGBoost domina las tres métricas.

XGBoost fue claramente el mejor modelo: 69,96 % de exactitud, F1 de 0,711 y AUC de 0,768, superando a Random Forest y a la Regresión Logística. El AUC por encima de 0,76 indica una capacidad discriminativa razonable considerando que las correlaciones lineales individuales con el retraso eran débiles.

5. Benchmark Polars vs Pandas

Se implementó exactamente el mismo pipeline con Pandas para comparar tiempos.

Información del sistema utilizado: 24 núcleos de CPU, 30,8 GB de RAM total (27,0 GB disponibles) y un dataset de trabajo de 90 299 filas. La Tabla 2 reporta los tiempos por operación y el speedup (tiempo Pandas / tiempo Polars).

Operación	Polars (s)	Pandas (s)	Speedup
Lectura del CSV	0.0143	0.2046	14.32×
Filtrado	0.0123	0.0156	1.27×
Agregación (group_by)	0.0106	0.0111	1.05×
Join	0.0035	0.0171	4.96×
Feature engineering	0.0143	0.0669	4.66×
Total del pipeline	0.0550	0.3154	5.73×

Tabla 2. Tiempos por operación y speedup de Polars sobre Pandas. El pipeline completo fue 5,73× más rápido en Polars.

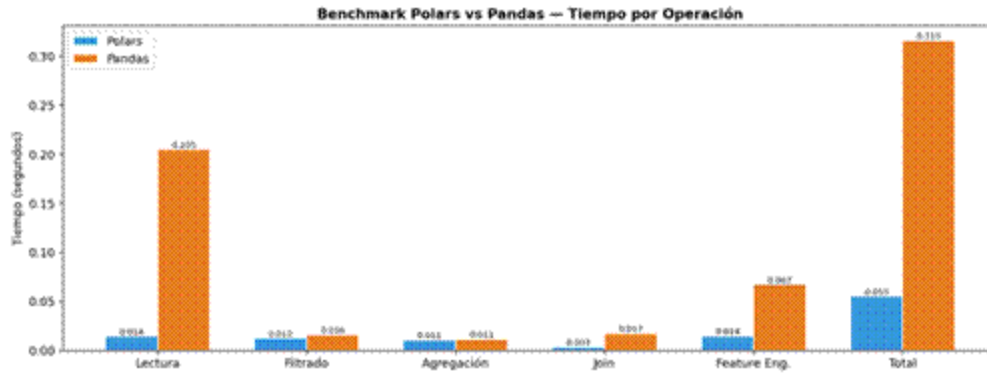


Figura 5. Tiempo por operación: Polars (azul) vs Pandas (naranja).

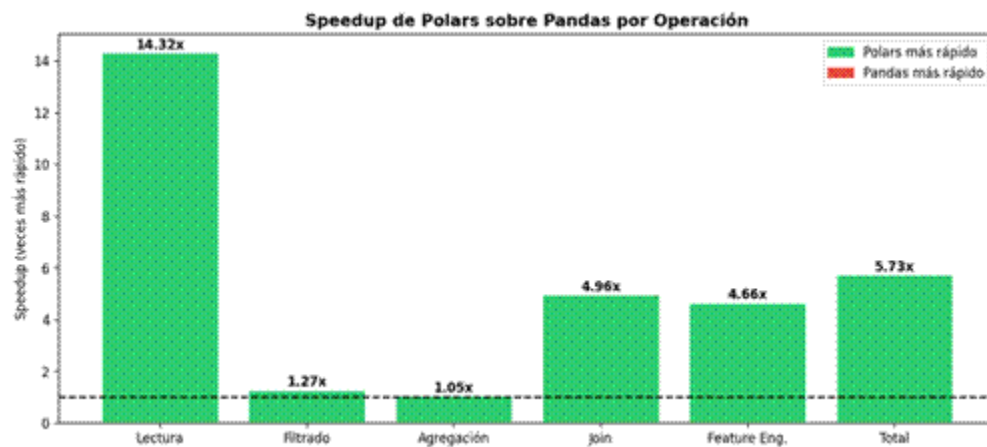


Figura 6. Speedup de Polars sobre Pandas por operación. La lectura del CSV es la operación con mayor ventaja (14,3×).

La mayor ventaja se observó en la lectura del CSV (14,32×), seguida del join (4,96×) y el feature engineering (4,66×). En operaciones simples como la agregación (1,05×) y el filtrado (1,27×) la diferencia fue pequeña, porque sobre este volumen de datos domina el overhead fijo y ambas bibliotecas son rápidas. En conjunto, el pipeline completo fue casi 6 veces más rápido en Polars.

6. Experimentos Adicionales

6.1 Escalabilidad con tamaño de datos

En este experimento se evaluó el tiempo total del pipeline empleando porciones del dataset equivalentes al 25 %, 50 %, 75 % y 100 %, con la finalidad de observar cómo evoluciona el desempeño de Polars en comparación con Pandas a medida que se incrementa el volumen de datos. Los datos obtenidos revelan que Pandas presentó un crecimiento más pronunciado en su tiempo de ejecución conforme aumentaba el tamaño del conjunto, mientras que Polars logró sostener tiempos notablemente inferiores en cada uno de los casos analizados.

Por su parte, el speedup de Polars creció de manera gradual, pasando de 1,82× con una cuarta parte del dataset hasta alcanzar 2,87× con el dataset completo. Este comportamiento demuestra que las ventajas de Polars se acentúan cuando el procesamiento involucra mayores cantidades de datos, pues su paralelismo interno y su manejo eficiente de la memoria le permiten distribuir mejor el costo fijo asociado al procesamiento. En síntesis, los resultados del experimento ratifican que Polars resulta más beneficioso en contextos donde tanto el volumen de información como la intensidad de las transformaciones son elevados.

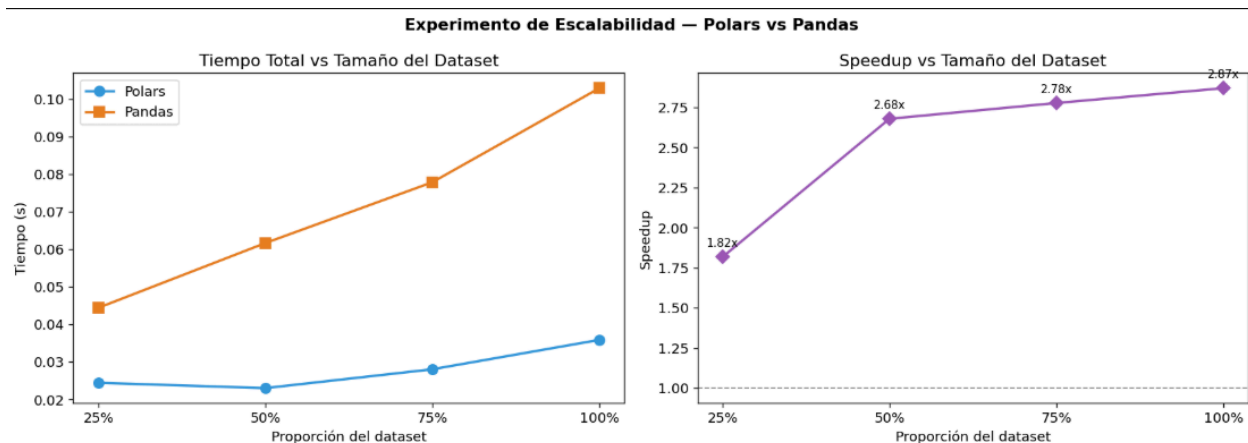


Figura 7. Experimento de Escalabilidad: comparación del tiempo total del pipeline y speedup de Polars frente a Pandas según el tamaño del dataset.

6.2 Lazy Execution

En este experimento se contrastó el modo eager, que emplea `pl.read_csv()` con ejecución inmediata de las operaciones, frente al modo lazy, basado en `pl.scan_csv(...).collect()`, con el propósito de cuantificar las diferencias en tiempo de ejecución y uso de memoria.

Los hallazgos indican que el modo lazy superó en velocidad al modo eager. El tiempo de procesamiento se redujo de 0,0344 a 0,0259 segundos, lo cual refleja una ganancia de eficiencia derivada de la optimización del plan de ejecución previa a la materialización de los datos. Este comportamiento corrobora una de las fortalezas centrales de Lazy Execution: la capacidad de Polars para reordenar y optimizar las operaciones antes de llevarlas a cabo.

No obstante, en esta ejecución el consumo de memoria resultó superior en el modo lazy, incrementándose de 17,88 MB en eager a 99,16 MB en lazy. Esto sugiere que, para este volumen de datos y esta operación en particular, la mejora en tiempo no se reflejó necesariamente en un ahorro de memoria. Es probable que el overhead vinculado a la construcción y materialización del plan lazy haya pesado más debido a la cantidad relativamente reducida de datos involucrados.

=== COMPARACIÓN EAGER vs LAZY ===		
Métrica	Eager	Lazy

Tiempo (s)	0.0344	0.0259
Memoria (MB)	17.88	99.16

Tabla 3. Comparación entre ejecución eager y lazy en Polars. El modo lazy redujo el tiempo de ejecución, aunque presentó mayor consumo de memoria en esta corrida.

7. Análisis de Resultados

1. ¿Qué ventajas observó al utilizar Polars?

Polars resultó sobretodo ser más eficiente que Pandas en cuanto al rendimiento computacional alcanzado dada su arquitectura. Estando basada en Apache Arrow, optimizó el tiempo de ejecución de cada consulta. Según los resultados solo la en la lectura del CSV fue 14 veces más rápida que la de pandas. Sin contar que, el pipeline completo alcanzó una aceleración de aproximadamente 5 veces, utilizando los 24 núcleos disponibles en el nodo del Kabré. Otra de

las ventajas que proporcionó Kabre fue la forma de expresar los datos, siendo legibles y fáciles de comprender, fue a primer momento junto a su velocidad lo que más nos llamó la atención.

2. ¿Qué operaciones obtuvieron el mayor speedup?

La de mayor relevancia fue la lectura del dataset con un rendimiento 14.32x, lo que podría explicarse porque Polars paraleliza la lectura del archivo haciendo uso de múltiples núcleos mientras que Pandas lo hace con solo uno. Después seguiría el join con un 4.96x, y el feature engineering con un 4.66x, haciéndose ver que Polars ya de por sí optimiza de forma interna el cómo se realizarán las ejecuciones antes de materializarlas.

3. ¿En cuáles operaciones la diferencia fue pequeña?

Aquellas operaciones con menor diferencia de speedup fueron la de agrupación por group_by con un aumento de 1.05x, y el filtrado que alcanzó un 1.27x. Con 90299 registros, es posible que no hubiese suficiente material como para poder visualizar la diferencia con mayor exactitud. Puede decirse que, al menos en estas operaciones, Pandas ha sido lo suficientemente eficiente como para que Polars no represente significativo aún considerando esos pequeños aumentos.

4. ¿Qué beneficios aporta Lazy Execution?

Entre sus beneficios está que permite a Polars realizar sus ejecuciones sin procesar ningún dato hasta que se utiliza la función `.collect()`. En vez de hacer cada operación cada vez que la llama, Polars acumula las transformaciones para optimizarlas y luego hacerlas ver. En el experimento realizado, el modo lazy superó el eager quedando 0.0196 frente a 0.0349 segundos. El problema aparece cuando se observa el consumo de memoria, siendo que en el lazy se ocuparon 99.16 megabytes frente a los 17.88 del eager. Esto puede significar que Polars de forma interna construye estructuras que representan dentro de la ejecución parte de la memoria hasta que llega al `.collect()`. Habría que evaluar su rendimiento con datasets de mayor tamaño.

5. ¿Qué limitaciones encontró en Polars?

En la elaboración del código, la ejecución con Polars requiere un mayor enfoque en los detalles, como a la hora de especificar los formatos de las transformaciones de tiempo que a primer momento puede parecer poco intuitivas. Además, para integrar scikit-learn al pipeline se necesitó de una integración externa para pasar el DataFrame por NumPy ya que Polar no es

compatible directamente con la API de scikit-learn. Esto último es algo de lo que podrían darse más casos ya que a diferencia de Pandas, Polars no está tan integrado al ecosistema de datos.

6. ¿Qué ventajas mantiene Pandas?

Pandas mantiene ventajas importantes por su madurez, estabilidad y amplia adopción dentro del ecosistema de ciencia de datos. Cuenta con documentación extensa, una gran cantidad de ejemplos disponibles y una comunidad muy activa. Además, su integración con bibliotecas como scikit-learn, matplotlib y seaborn sigue siendo directa y ampliamente soportada, lo que facilita el desarrollo de análisis exploratorios, visualizaciones y modelos de aprendizaje automático.

7. ¿La aceleración observada justifica migrar un proyecto existente?

La migración a Polars se justifica principalmente en proyectos donde el procesamiento de datos representa una parte significativa del tiempo total de ejecución. En este caso, el pipeline completo fue aproximadamente 5,7 veces más rápido y la lectura del CSV superó a Pandas por más de 14 veces, lo cual representa una mejora considerable en escenarios de ETL intensivo. Sin embargo, para análisis pequeños, exploratorios o de ejecución puntual, la ventaja absoluta puede ser menor y no necesariamente compensar el esfuerzo de migración, especialmente si el proyecto depende fuertemente de integraciones propias del ecosistema de Pandas.

8. ¿Cómo afecta el tamaño del dataset al beneficio obtenido?

Se espera que el beneficio de Polars aumente conforme crece el tamaño del dataset, ya que su arquitectura columnar, el paralelismo interno y la gestión eficiente de memoria permiten aprovechar mejor los recursos disponibles cuando el volumen de datos es mayor. En datasets pequeños, parte de la ventaja puede verse limitada por el overhead fijo de ejecución. No obstante, la medición de escalabilidad quedó pendiente de ejecución, por lo que esta conclusión debe tomarse como una expectativa técnica y no como un resultado experimental confirmado dentro de esta corrida.

9. ¿Qué modelo produjo el mejor desempeño predictivo?

El modelo con mejor desempeño predictivo fue XGBoost, con una exactitud de 69,96 %, un F1 Score de 0,711 y un AUC de 0,768. Estos resultados superaron los obtenidos por Random Forest y Regresión Logística. Además, XGBoost presentó el menor tiempo de entrenamiento entre los tres modelos evaluados, lo que evidencia un buen balance entre capacidad predictiva y eficiencia computacional para este problema de clasificación binaria.

10. ¿Qué recomendaciones daría para proyectos futuros?

Para proyectos futuros, se recomienda utilizar Polars en las etapas de ETL, especialmente cuando se trabaje con volúmenes grandes de datos o pipelines con múltiples transformaciones, joins y agregaciones. También sería conveniente ejecutar el pipeline en modo Lazy para evaluar posibles mejoras adicionales en tiempo y memoria. Para mantener compatibilidad con el ecosistema de Machine Learning, se puede convertir el resultado final a NumPy o Pandas mediante `to_numpy()` o `to_pandas()` antes del entrenamiento.

8. Discusión Técnica

Los resultados obtenidos demuestran que Polars aporta mejoras significativas en las fases de lectura, escritura y transformación de datos, etapas que constituyen componentes esenciales dentro de un pipeline de ciencia de datos. La carga del archivo CSV fue la operación donde se registró la mayor brecha respecto a Pandas, alcanzando una aceleración superior a 14 veces. De igual forma, el pipeline en su totalidad resultó casi 6 veces más veloz, con avances destacados en operaciones como joins y feature engineering.

Pese a ello, la superioridad de Polars no se manifestó de manera homogénea en todas las operaciones. En tareas sencillas como el filtrado y la agregación, la brecha fue más reducida, probablemente debido a que el volumen del dataset no alcanzó la magnitud necesaria para que el paralelismo interno marcara una diferencia más notoria. En estos escenarios, el overhead fijo de ejecución tiende a atenuar el impacto perceptible del motor paralelo.

En el plano del modelado, el desempeño se estabilizó en torno al 70 % de exactitud. XGBoost alcanzó el mejor resultado global, lo cual concuerda con su habilidad para representar relaciones no lineales en datos tabulares. Aun así, el rendimiento no superó dicho umbral porque la predicción de retrasos está condicionada por factores externos ausentes en el dataset, tales como las condiciones meteorológicas, la saturación del espacio aéreo, la disponibilidad operativa, las conexiones entre vuelos y los efectos en cascada.

La matriz de correlación reveló que las variables numéricas disponibles mantienen correlaciones lineales débiles con el retraso. Ello justifica el desempeño acotado de los modelos lineales y la mayor eficacia de los modelos basados en árboles. Las variables relativas al retraso histórico promedio por aeropuerto y por ruta proporcionaron señal predictiva valiosa, aunque insuficiente para abarcar toda la complejidad del fenómeno.

En síntesis, los resultados confirman que Polars resulta particularmente provechoso para agilizar el procesamiento previo al modelado, mientras que la mejora en la capacidad predictiva depende en mayor medida de la calidad y diversidad de las variables disponibles que de la biblioteca empleada para el ETL.

9. Conclusiones

Se desarrolló con éxito un pipeline integral de análisis de datos y aprendizaje automático mediante Polars para la predicción de retrasos en vuelos. El dataset inicial reunía 107 833 registros y, tras los procesos de limpieza, filtrado y eliminación de valores atípicos, se conformó un conjunto definitivo de 90 299 registros con clases balanceadas.

La comparación con Pandas evidenció ventajas nítidas de Polars en cuanto a rendimiento computacional. La lectura del CSV resultó aproximadamente 14,3 veces más rápida y el pipeline completo logró una aceleración de 5,7 veces. Los mayores beneficios se concentraron en la lectura, los joins y el feature engineering, en tanto que las diferencias fueron más modestas en operaciones simples como el filtrado y la agregación.

Durante la fase de Machine Learning, XGBoost se posicionó como el modelo de mejor desempeño entre los evaluados, con una exactitud del 69,96 %, un AUC de 0,768 y el menor tiempo de entrenamiento. Esto evidencia que, para este conjunto de datos, XGBoost alcanzó el equilibrio más favorable entre calidad predictiva y eficiencia.

Se concluye que Polars constituye una opción recomendable para proyectos de ciencia de datos con una carga considerable de ETL, sobre todo al trabajar con datasets de gran tamaño o transformaciones repetitivas. Pandas, en cambio, sigue siendo una herramienta de gran valor por su madurez, su documentación y su integración con el ecosistema tradicional de análisis y modelado.

Como línea de trabajo futura, se sugiere ejecutar los experimentos de escalabilidad y Lazy Execution ya disponibles en el notebook, con el propósito de completar el análisis de rendimiento y verificar de forma experimental cómo evoluciona el speedup de Polars a medida que crece el tamaño del dataset y se optimiza el plan de ejecución.

10. Enlaces

GitHub: https://github.com/Nokthu/tarea3_polars